

ONLINE TUTORING SESSION BOOKING SYSTEM

Project Report

An Express.js, MongoDB and Mongoose based REST API
with JWT Authentication and Role-Based Authorization

Project Details	Information
Project Title	Online Tutoring Session Booking System
Technology	Node.js, Express.js, MongoDB, Mongoose
Authentication	JWT (JSON Web Token)
Database	MongoDB / MongoDB Atlas
API Testing	Postman / Thunder Client
Frontend (Suggested)	React.js

Prepared for Academic Submission

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the System
5. Proposed Solution
6. System Architecture
7. Technology Stack
8. Database Design
9. Mongoose Schemas
10. Authentication and Authorization
11. Core API Endpoints
12. Session Booking and Conflict Validation
13. Tutor Availability Management
14. Data Validation
15. Suggested React Frontend
16. Testing Strategy
17. Environment Configuration
18. Deployment Plan
19. Security Considerations
20. Expected Outcomes
21. Future Enhancements
22. Conclusion

1. Introduction

The Online Tutoring Session Booking System is a web-based application designed to simplify one-on-one tutoring session scheduling. The platform allows students to search for tutors according to subject and availability and book a suitable time slot. Tutors can publish their subjects and available time slots, while the backend ensures that the same tutor cannot be assigned overlapping sessions.

The system is implemented as an Express.js REST API connected to MongoDB through Mongoose. JWT authentication is used to identify users, and role-based authorization separates student operations from tutor-only operations. Scheduling and conflict validation remain on the backend so that the rules cannot be bypassed by a frontend client.

2. Problem Statement

An ed-tech platform wants students to book one-on-one tutoring sessions with tutors based on subject and availability. A tutor must not have two sessions booked at overlapping times, and a student should not be able to book the same tutor twice for the same slot. The system therefore requires secure authentication, role-based access, validated scheduling data, and reliable conflict checking.

3. Objectives

- Design Student, Tutor and Session schemas using Mongoose.
- Implement JWT authentication for students and tutors.
- Allow tutors to publish their subjects and available time slots.
- Allow students to book tutoring sessions only when the requested slot is genuinely free.
- Prevent overlapping bookings for tutors.
- Prevent duplicate bookings for a student with the same tutor and slot.
- Implement role-based authorization for tutor-only availability management.
- Validate required session and scheduling fields.
- Provide REST APIs that can be tested using Postman or Thunder Client.
- Prepare the system for deployment using MongoDB Atlas and a cloud backend.

4. Scope of the System

The system covers the following major functions:

- Student registration and login.
- Tutor registration and login.
- JWT-based protected API access.
- Tutor subject and availability management.
- Tutor search based on subject and available slots.
- Student session booking.

- Session conflict detection.
- Student and tutor session dashboards.
- REST API testing through Postman or Thunder Client.
- Environment-based configuration for database connection and JWT secret.

5. Proposed Solution

The proposed solution follows a client-server architecture. A React frontend, if implemented, communicates with the Express.js backend using HTTP requests. The backend authenticates users, checks their roles, validates request data, performs scheduling conflict checks, and stores data in MongoDB.

User Role	Main Responsibilities
Student	Register/login, search tutors, view slots, book sessions, view own sessions.
Tutor	Register/login, manage subjects, publish availability, view booked sessions.
Backend	Authentication, authorization, validation, conflict checks, database operations.
Database	Stores students, tutors, sessions and their relationships.

6. System Architecture

The system can be represented using four logical layers:

- Presentation Layer: React.js pages such as Login/Register, Tutor Search, Booking Form and My Sessions.
- API Layer: Express.js routes receive HTTP requests and return JSON responses.
- Business Logic Layer: Authentication, role authorization, input validation and session conflict logic.
- Data Layer: Mongoose models communicate with MongoDB collections.

Basic request flow:

React Client → Express Route → JWT Middleware → Validation → Business Logic → Mongoose → MongoDB → JSON Response

7. Technology Stack

Technology	Purpose
Node.js	JavaScript runtime for backend development.
Express.js	REST API and route handling.
MongoDB	NoSQL database for application data.
Mongoose	Schema definition, models and MongoDB interaction.
JWT	Authentication and secure API access.
bcrypt	Password hashing before storing user credentials.
React.js	Suggested frontend interface.
Axios / Fetch	Frontend communication with REST APIs.
Postman / Thunder Client	API testing.
MongoDB Atlas	Cloud-hosted MongoDB deployment.
Render / Railway	Suggested backend deployment platforms.
Vercel / Netlify	Suggested frontend deployment platforms.

8. Database Design

The database uses referenced collections. Students and tutors are maintained as separate collections, while sessions reference the student and tutor documents through MongoDB ObjectIds.

Collection	Important Fields	Relationship
students	name, email, password, role	Referenced by Session
tutors	name, email, password, role, subjects[], availableSlots[]	Referenced by Session
sessions	student, tutor, subject, startTime, endTime, status	References Student and Tutor

9. Mongoose Schemas

9.1 Student Schema

The Student schema stores account information required for authentication and session ownership. The password should be stored as a secure hash rather than plain text.

Field	Type	Purpose
name	String	Student's name.
email	String	Unique login identifier.

password	String	Hashed password.
role	String	Set to student.

9.2 Tutor Schema

The Tutor schema stores the tutor profile, subjects offered and published availability.

Field	Type	Purpose
name	String	Tutor's name.
email	String	Unique login identifier.
password	String	Hashed password.
role	String	Set to tutor.
subjects	Array of String	Subjects taught by the tutor.
availableSlots	Array of Objects	Start and end times offered for booking.

9.3 Session Schema

The Session schema represents a booked tutoring session and references both the student and tutor.

Field	Type	Purpose
student	ObjectId	Reference to Student.
tutor	ObjectId	Reference to Tutor.
subject	String	Subject for the session.
startTime	Date	Session start time.
endTime	Date	Session end time.
status	String	Booking status such as booked or cancelled.
createdAt	Date	Time the booking was created.

10. Authentication and Authorization

JWT authentication is used to protect private API endpoints. After successful login, the server generates a signed token containing the user's identity and role. The client sends the token in the Authorization header for protected requests.

Authorization header: Authorization: Bearer <JWT_TOKEN>

Authentication confirms who the user is. Authorization determines what the authenticated user is allowed to do. For example, a tutor can create availability slots, while a student cannot access the tutor-only slot management endpoint.

11. Core API Endpoints

Method	Endpoint	Access	Purpose
POST	/auth/register/student	Public	Register a student.
POST	/auth/register/tutor	Public	Register a tutor.
POST	/auth/login	Public	Login and receive JWT.
GET	/tutors	Authenticated	Search/list tutors.
GET	/tutors/:id	Authenticated	View tutor details.
POST	/tutors/:id/slots	Tutor only	Publish an available slot.
GET	/tutors/:id/slots	Authenticated	View tutor availability.
POST	/sessions/book	Student only	Book a tutoring session.
GET	/sessions/my	Authenticated	View user's sessions.
PATCH	/sessions/:id/cancel	Owner/authorized	Cancel a booking.

12. Session Booking and Conflict Validation

Conflict validation is the central business rule of the application. The backend must not rely only on the frontend to determine whether a slot is free. Before creating a session, the Express server checks the database for conflicting sessions.

12.1 Tutor Conflict

A new booking conflicts with an existing tutor booking when the time ranges overlap. For two intervals A and B, an overlap exists when:

$$\text{newStart} < \text{existingEnd} \text{ AND } \text{newEnd} > \text{existingStart}$$

If this condition is true for any active session of the tutor, the booking is rejected.

12.2 Student Conflict

The same overlap rule is applied to the student's existing sessions. This prevents the student from booking another session that occurs at the same time.

12.3 Same Tutor and Same Slot

The backend also checks whether the student has already booked the same tutor for the requested slot. A duplicate booking request should return an appropriate client error rather than creating another session.

12.4 Booking Flow

- Student logs in and receives a JWT.
- Student selects a tutor, subject, start time and end time.
- Request is sent to POST /sessions/book.
- JWT middleware verifies the student.

- Request validation checks required fields and valid date order.
- Backend verifies that the tutor exists and teaches the requested subject.
- Backend verifies that the requested time is within a tutor-published available slot.
- Backend checks tutor schedule for overlapping active sessions.
- Backend checks student schedule for overlapping active sessions.
- Backend checks for duplicate tutor-slot booking.
- If all checks pass, the session is saved in MongoDB.
- The API returns the created session and booking confirmation.

13. Tutor Availability Management

Only authenticated tutors should be allowed to create or modify their own availability. The role-based middleware verifies the JWT role before the route handler executes.

- Tutor logs in.
- JWT identifies the account and role as tutor.
- Tutor sends a request to POST /tutors/:id/slots.
- Authorization middleware checks the role.
- The backend verifies that the route ID belongs to the authenticated tutor.
- The slot is validated for valid start/end times.
- The slot is added to the tutor's availableSlots array.

14. Data Validation

- Name and email are required during registration.
- Email should be unique and correctly formatted.
- Password must meet the application's minimum security requirements.
- Tutor subject names cannot be empty.
- Session must contain a valid tutor, student, subject, start time and end time.
- End time must be later than start time.
- Date and time values should use a consistent timezone strategy.
- Only supported session status values should be accepted.
- A student cannot create a session using another student's identity.
- A tutor cannot manage another tutor's availability.

15. Suggested React Frontend

A basic React frontend can be connected to the backend after the REST API is completed. The frontend is responsible for presenting information and sending API requests; the actual booking and conflict rules remain in Express.js.

Page	Main Functions
Login / Register	Create account and authenticate users.
Tutor Search	Search tutors by subject and inspect available slots.
Booking Form	Select subject, date, time and submit a booking request.
My Sessions	Display upcoming and previous sessions.
Tutor Dashboard	Manage available slots and view booked sessions.

16. Testing Strategy

Postman or Thunder Client can be used to test the REST API. Testing should include both successful requests and invalid requests to confirm that the business rules cannot be bypassed.

Test Case	Expected Result
Register valid student	Student account created successfully.
Register duplicate email	Request rejected.
Login with valid credentials	JWT token returned.
Login with invalid credentials	Authentication rejected.
Tutor creates valid slot	Slot added successfully.
Student attempts to create tutor slot	403 Forbidden / unauthorized response.
Book free slot	Session created successfully.
Book overlapping tutor slot	Booking rejected due to tutor conflict.
Book overlapping student slot	Booking rejected due to student conflict.
Book same tutor and same slot twice	Duplicate booking rejected.
Invalid end time	Validation error returned.
Missing required field	Validation error returned.

17. Environment Configuration

Sensitive configuration values should not be hard-coded in the source code. They should be stored in environment variables and excluded from version control.

Example .env configuration:

```
PORT=5000
```

```
MONGO_URI=<MongoDB_Atlas_Connection_String>
```

JWT_SECRET=<Strong_Secret_Key>

18. Deployment Plan

The backend can be deployed on Render or Railway. MongoDB Atlas can provide the cloud database. The frontend can be deployed on Vercel or Netlify.

Component	Suggested Platform	Important Configuration
MongoDB	MongoDB Atlas	Database URI and network access.
Backend	Render / Railway	PORT, MONGO_URI and JWT_SECRET.
Frontend	Vercel / Netlify	API base URL environment variable.

The React application should use an environment-based API base URL so that the same frontend code can communicate with a local development server and the deployed backend.

19. Security Considerations

- Passwords must be hashed using a secure password-hashing library such as bcrypt.
- JWT secrets must be stored in environment variables.
- Protected endpoints must verify the JWT before processing private operations.
- Role-based authorization must be enforced on the server.
- Users must only be allowed to access or modify resources they are authorized to use.
- Input data should be validated before database operations.
- MongoDB connection credentials must never be committed to a public repository.
- Production deployments should use HTTPS.
- Expired, invalid or malformed JWTs should be rejected.
- Database indexes and suitable query patterns should be considered for high-volume scheduling.

20. Expected Outcomes

- Tutors can publish their available time slots by subject.
- Students can search for tutors and view available slots.
- Students can book only genuinely free sessions.
- Tutors cannot receive overlapping session bookings.
- Students cannot create conflicting bookings in their own schedule.
- Unauthorized users cannot modify tutor availability.
- Session data is stored using MongoDB references between students and tutors.

- The REST API can be tested independently using Postman or Thunder Client.
- The application is structured for deployment using cloud services.

21. Future Enhancements

- Online payment integration for paid tutoring sessions.
- Email or SMS booking confirmations and reminders.
- Tutor ratings and student reviews.
- Calendar integration such as Google Calendar.
- Video meeting integration for online tutoring.
- Tutor verification and profile approval.
- Advanced tutor search using subject, language, rating and price.
- Admin dashboard for platform management.
- Recurring tutoring sessions.
- Automatic timezone conversion for students and tutors in different regions.

22. Conclusion

The Online Tutoring Session Booking System provides a structured solution for scheduling one-on-one online tutoring sessions. The combination of Express.js, Mongoose and MongoDB provides a flexible REST backend, while JWT authentication and role-based authorization secure user operations. The most important part of the system is server-side conflict validation, which ensures that tutors do not receive overlapping bookings and students cannot create conflicting or duplicate sessions.

The proposed architecture also keeps the frontend independent from the core business rules. This allows the same backend API to support a React application, mobile client or other future interfaces while maintaining consistent authentication, authorization and booking logic.

Appendix A: Suggested Project Structure

```
online-tutoring-booking/online-tutoring-booking/
├── config/ └── config/
│   ├── db.js └── db.js
├── middleware/ └── middleware/
│   ├── authMiddleware.js └── authMiddleware.js
│   └── roleMiddleware.js └── roleMiddleware.js
├── models/ └── models/
│   ├── Student.js └── Student.js
│   ├── Tutor.js └── Tutor.js
│   └── Session.js └── Session.js
├── routes/ └── routes/
│   ├── authRoutes.js └── authRoutes.js
│   └── tutorRoutes.js └── tutorRoutes.js
```

```

|   └─ sessionRoutes.js |   └─ sessionRoutes.js
└─ controllers/ └─ controllers/
|   └─ authController.js |   └─ authController.js
|   └─ tutorController.js |   └─ tutorController.js
|   └─ sessionController.js |   └─ sessionController.js
└─ .env └─ .env
└─ .gitignore └─ .gitignore
└─ server.js └─ server.js
└─ package.json └─ package.json
└─ README.md └─ README.md

```

Appendix B: Sample Booking Request

POST /sessions/book

```

{
  "tutorId": "<tutor_id>",
  "subject": "JavaScript",
  "startTime": "2026-10-10T10:00:00.000Z",
  "endTime": "2026-10-10T11:00:00.000Z"
}

```

— End of Report —